Multimodal visual positioning application

Multimodal visual positioning application

1. Concept introduction
    1.1 What is "multimodal visual positioning"?
    1.2 Brief description of the implementation principle
2. Code analysis
    Key code
        1. The tool layer entry (`largemodel/utils/tools_manager.py`)
        2. The model interface layer (`largemodel/utils/large_model_interface.py`)
    Code analysis
3. Hands-on operation
    3.1 Configure the large model
        3.1.1 Configure the LLM platform (`yahboom.yaml`)
        3.1.2 Configure the model interface (`large_model_interface.yaml`)
    3.2 Start it and test the function (text mode)
    3.3 Voice interaction
4. Frequently asked questions and solutions
        Problem 1: it reports "the image file cannot be found"

> **Hardware requirement: a Jetson with a 4GB configuration cannot run the cases of this tutorial; please use a Jetson with 8GB of memory or more.**

---

1. Concept introduction

1.1 What is "multimodal visual positioning"?

Multimodal visual positioning is a technology that combines multiple sensor inputs (such as the camera, the depth sensor, the IMU and so on) with algorithmic processing to recognise and track the precise position and pose of a device or a user in the environment. This technology does not rely on a single type of sensor data; instead it integrates the information coming from the different perception modes, which improves the accuracy and the robustness of the positioning.

1.2 Brief description of the implementation principle

1. **Cross-modal representation learning**: so that the LLM is able to process visual information, a mechanism has to be developed that converts the visual signal into a form the model can understand. This may involve extracting the features with a convolutional neural network (CNN) or another architecture suited to image processing and mapping them into the same embedding space as the text.
2. **Joint training**: by designing the appropriate loss functions, the text and the visual data can be trained within the same framework, so that the model learns to build the connection between the two modes. For example, in a question answering system it can give the answer either from the provided text question or from the content of the related picture.
3. **Vision guided language generation/understanding**: once an effective cross-modal representation has been established, the visual information can be used to enhance the capabilities of the language model. For example, when it is given a photo, the model can not only describe what happens in the picture, but

also answer specific questions about the scene and even execute instructions based on the visual cues (such as navigating to a certain place).

---

2. Code analysis

Key code

1. The tool layer entry (`largemodel/utils/tools_manager.py`)

The `visual_positioning` function in this file defines the execution flow of the tool, in particular how it builds a Prompt that contains the name of the target object and the format requirements.

```
# From largemodel/utils/tools_manager.py
class ToolsManager:
    # ...
    def visual_positioning(self, args):
        """
        Locate the object coordinates in the image and save the results to an MD file.

        :param args: Arguments containing image path and object name.
        :return: Dictionary with file path and coordinate data.
        """
        self.node.get_logger().info(f"Executing visual_positioning() tool with args:
{args}")
        try:
            image_path = args.get("image_path")
            object_name = args.get("object_name")
            # ... (the path fallback mechanism and the parameter check)

            # Construct a prompt asking the large model to identify the coordinates of the
specified object.
            if self.node.language == 'zh':
                prompt = f"Please carefully analyze this image and use a box for each one to
locate the position of every {object_name} in the image..."
            else:
                prompt = f"Please carefully analyze this image and find the position of all
{object_name}..."

            # ... (building the independent message context)

            result = self.node.model_client.infer_with_image(image_path, prompt,
message=message_to_use)

            # ... (processing and parsing the returned coordinate text)

            return {
                "file_path": md_file_path,
                "coordinates_content": coordinates_content,
                "explanation_content": explanation_content
            }
        # ... (the error handling)
```

2. The model interface layer (`largemodel/utils/large_model_interface.py`)

The `infer_with_image` function in this file is the unified entry point for all the image related tasks.

```
# From largemodel/utils/large_model_interface.py

class model_interface:
    # ...
    def infer_with_image(self, image_path, text=None, message=None):
        """Unified image inference interface."""
        # ... (preparing the message)
        try:
            # Depending on the value of self.llm_platform, decide which implementation to
call
            if self.llm_platform == 'ollama':
                response_content = self.ollama_infer(self.messages, image_path=image_path)
            elif self.llm_platform == 'tongyi':
                # ... (the logic for calling the Tongyi model)
                pass
            # ... (the logic of the other platforms)
        # ...
        return {'response': response_content, 'messages': self.messages.copy()}
```

Code analysis

The core of the visual positioning function is that it **guides the large model into outputting structured data through precise instructions**. It follows the same layered design of the tool layer and the model interface layer.

1. **The tool layer (`tools_manager.py`)**:
    - The `visual_positioning` function is the business core of this function. It receives two key parameters: `image_path` (the path of the image) and `object_name` (the name of the object to be located).
    - The most core operation of this function is **building a highly customised Prompt**. It does not simply ask the model to describe the picture; it embeds `object_name` into a carefully designed template that explicitly asks the model to "locate the position of every {object_name} in the image", and it implicitly or explicitly requires the result to be returned in a specific format (such as an array of coordinates).
    - Once the Prompt has been built, it calls the `infer_with_image` method of the model interface layer and passes the image together with this customised instruction.
    - After it receives the returned text from the model interface layer, it still has to do some **post-processing**: it parses the precise coordinate data out of the natural language reply of the model with methods such as regular expressions.
    - Finally, it returns the parsed structured coordinate data to the application above.
2. **The model interface layer (`large_model_interface.py`)**:
    - The `infer_with_image` function still plays the role of the "dispatch centre". It receives the image and the Prompt from `visual_positioning` and dispatches the task to the correct backend model implementation according to the current configuration (`self.llm_platform`).

- For the visual positioning task, the responsibility of the model interface layer is basically the same as for the visual understanding task: it packs the image data and the text instruction correctly, sends them to the selected model platform and then hands the returned text result back to the tool layer unchanged. All the platform specific implementation details are encapsulated in this layer.

In summary, the general flow of the visual positioning is: `ToolsManager` receives the name of the target object and builds a precise Prompt that asks for the coordinates -> `ToolsManager` calls the model interface -> `model_interface` packs the image and that Prompt together and sends it to the corresponding model platform according to the configuration -> the model returns the text that contains the coordinate information -> `model_interface` returns the text to `ToolsManager` -> `ToolsManager` parses the text, extracts the structured coordinate data and returns it. This process shows how the Prompt Engineering technique lets a general purpose vision large model complete a more specific and more structured task.

3. Hands-on operation

3.1 Configure the large model

This section selects the backend through the platform option in `yahboom.yaml` and fills in the corresponding model parameters in `large_model_interface.yaml`. The online example uses Tongyi Qianwen; the offline example uses the local Ollama.

3.1.1 Configure the LLM platform (`yahboom.yaml`)

Open the main configuration file:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

Set `llm_platform` in the existing `model_service.ros__parameters` parameter block. You only have to choose one of the two backends:

```
model_service:
  ros__parameters:
    language: 'en'
    llm_platform: 'ollama'  # The offline large model; change it to 'tongyi' when you use
the online model
    regional_setting: "international"
```

- Offline Ollama: set `llm_platform: 'ollama'`.
- Online Tongyi Qianwen: set `llm_platform: 'tongyi'`.

If these parameters already exist in the file, only modify the corresponding values and keep the other configuration; do not add duplicate YAML keys with the same name.

3.1.2 Configure the model interface (`large_model_interface.yaml`)

Open the model interface configuration file:

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

Depending on the backend selected in the previous step, only fill in the matching group of parameters.

Online Tongyi Qianwen: first apply for the API Key as described in the online large model basic tutorial and confirm that the account has available quota; then fill in the Key and a model that supports image input:

```
qianwen_api_key: "Replace it with your own API-Key"
qianwen_model: "qwen-vl-max-latest"
```

Replace the placeholder with your own API Key. The visual positioning needs a model that understands the content of the image and that can return the position or the coordinates of the target following the prompt. Never publish a real API Key.

Offline Ollama: make sure the Ollama service is running and that a model supporting image input has been downloaded; then fill in the service address and the name of the local model:

```
ollama_host: "http://localhost:11434"
ollama_model: "llava"  # Replace it with the model name from ollama list
```

The model name has to match the output of `ollama list`, and the model has to support image input.

After saving both configuration files, restart `largemodel` so that the settings take effect. The online mode requires the Jetson to be able to reach the Tongyi service; the offline mode requires the local Ollama service to be available.

> The speed of the visual positioning inference of the offline large model depends on the computing power of the Jetson mainboard and is usually slower than the online large model; the actual speed also depends on the size of the model and on the input image.

3.2 Start it and test the function (text mode)

1. **Prepare the image file**:

   Put an image file to be tested at the following path:
   `/home/jetson/yahboom_ws/src/largemodel/resources_file/visual_positioning`

   Then name the image `test_image.jpg`

2. **Start the `largemodel` main program (text mode)**:

   Open a terminal and run the following command:

   ```
   ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=true
   ```

3. **Send the text instruction**:
   Open another terminal and run the following command,

   ```
   ros2 run text_chat text_chat
   ```

   Then start typing the text: "locate the position of the dinosaur in the picture".

4. **Watch the result**:
   In the first terminal that runs the main program you will see the log output showing that the system

received the instruction, called the `visual_positioning` tool, reported that the visual_positioning execution completed and saved the coordinates to a document.

You can find this document at the path ~/yahboom_ws/src/largemodel/resources_file/visual_positioning

---

3.3 Voice interaction

The voice mode can use the online or the offline LLM. In `yahboom.yaml`, `llm_platform` is set to `tongyi` for the online and to `ollama` for the offline; the ASR/TTS are chosen independently:

```
model_service:
  ros__parameters:
    use_oline_asr: False  # True: online ASR; False: offline ASR
    useolinetts: False    # True: online TTS; False: offline TTS
```

Complete the corresponding configuration as described in the ASR/TTS tutorials. Put the image in the directory specified above and name it `test_image.jpg`, and then start it:

```
ros2 launch largemodel largemodel_control.launch.py
```

After waking the device up, say "locate the position of the dinosaur in the picture". Check the recognised text of the ASR, the log of the `visual_positioning` tool call and the output document. The voice tool call depends on the current version of the program and on the capability of the model; if there is no tool call log, verify it with the text mode first.

4. Frequently asked questions and solutions

Problem 1: it reports "the image file cannot be found"

Solution:

1. **Check the path**: make sure you have placed the image file at the specified path, named it `test_image.jpg` and that you have the permission to read the file.
2. **The image format**: make sure the image file is not corrupted and that it is in the .jpg format.